From a Blank Computer to Your Own Local AI — the Whole Path

This is a start-to-finish build course. It begins with a Windows PC that has nothing installed and ends with you having built, by hand and for free: a local AI model running on your own machine, a chat client you wrote yourself, tools ("hands") with a real security boundary, and a windowed GUI. Nothing here requires an account, an API key, or a dollar.

Rules of the course:

- **Type every command yourself.** Every command states what it should do BEFORE you run it. If what happens doesn't match, stop there — that mismatch is the lesson.
- **Every code listing is complete and runnable.** No "..." hiding the hard part. (The `samples\` folder has each finished file, but you learn more typing them.)
- Build in a scratch folder like `C:\learn_ai\`. Nothing here touches the rest of your machine.

PART 0 — The whole machine in one picture

Three layers, and only one of them is yours:

```
+-------------------------------------------------------------+
|  YOUR CLIENT (the part you write: chat loop, hands, GUI)    |
|     talks HTTP to ->                                        |
+-------------------------------------------------------------+
|  INFERENCE SERVER (Ollama / llama.cpp -- someone else wrote)  |
|     loads and runs ->                                       |
+-------------------------------------------------------------+
|  MODEL WEIGHTS (a big file of numbers -- downloaded, frozen)  |
+-------------------------------------------------------------+
```

- **The weights** are a multi-gigabyte file of numbers. They don't "run" any more than an MP3 plays itself. They are the brain in the jar.
- **The inference server** (Ollama is the one we use) loads the weights into GPU/RAM and exposes them as a local web service: send text in, get text out. It is the life support and the phone line.
- **Your client** is a program that makes HTTP calls to that service. Everything that makes a local AI *yours* — the personality, the tools, the consent gates, the window — lives in this layer. The model never gets smarter or more dangerous because of your client; **your client decides what the model's words are allowed to cause.**

That last sentence is the entire security model of this course, one sentence long.

PART 1 — Vocabulary (short, blunt definitions)

- **Open-weight model** — the weights file is published so anyone can download and run it locally. "Open weights" != "open source": the training data and recipe usually stay private.
- **Parameters (7B, 24B...)** — how many numbers are in the brain. More = smarter and slower, roughly.
- **Quantization (Q4, Q8...)** — storing those numbers with fewer bits to shrink the file. In practice Q4 is the standard tradeoff for local use.
- **Dense vs MoE** — a dense model uses ALL its parameters for every word; a Mixture-of-Experts (MoE) model has many "experts" and activates only a few per word. That's how a 120B MoE can respond as fast as a much smaller dense model.

- **GGUF** — the file format llama.cpp-family servers read. One model = one .gguf.
- **Token** — the unit models actually read/write; ~3/4 of an English word.
- **Context window** — how many tokens the model can hold at once. Your entire conversation is re-sent every turn and must fit inside it.
- **System prompt** — the standing instructions the model sees before the conversation starts.
- **Loopback / 127.0.0.1** — the network address that never leaves the machine. A server bound to loopback is physically unreachable from your network.
- **API** — an agreed URL + JSON shape. Ollama's is `POST /api/chat`. Everything in this course is that one call, dressed up.

PART 2 — Install day (fresh Windows box)

Two installs. That's the whole shopping list — the client you'll write needs nothing but Python's standard library.

1. Python. Download from python.org (3.12+), run the installer, and CHECK "Add python.exe to PATH" on the first screen. Tkinter (the GUI toolkit) ships inside — no extra install. Verify in a fresh PowerShell:

```
python --version
```

Expected: it prints `Python 3.x.x` and nothing else. If "not recognized," PATH wasn't set — rerun the installer and use its Repair/modify option.

2. Ollama. Download the installer from ollama.com, run it. What it actually does: installs a background service listening on `http://127.0.0.1:11434` (loopback only, by default) and an `ollama` command-line tool. Verify:

```
ollama --version
```

Expected: prints a version number. The service auto-starts; if a later step says "connection refused," run `ollama serve` in its own window and leave it open.

PART 3 — Pick the model that fits YOUR card

This is where most people go wrong: they pull the biggest model they've heard of, it doesn't fit their GPU, everything crawls, and they conclude local AI is slow. Local AI isn't slow. **A model that doesn't fit is slow.**

Step 1 — find out how much VRAM you have

Task Manager -> Performance tab -> GPU -> "Dedicated GPU memory." Or in PowerShell:

```
nvidia-smi
```

Expected (NVIDIA cards): a table whose memory column shows something like `0MiB / 16384MiB`. The number after the `/` is your VRAM. No NVIDIA card or the command isn't found? You're in the CPU-only tier below — still workable.

Step 2 — the fitting rule

- A **Q4 model needs roughly 0.6–0.7 GB of VRAM per billion parameters**, plus 1–4 GB for context. An 8B model ~ 5 GB. A 24B ~ 14 GB. A 70B ~ 42 GB.
- **If it doesn't fit, the server "spills" layers to system RAM** and a dense model gets 5–10× slower. That cliff is the whole game. MoE models degrade far more gracefully when they spill (a modest tax, not a 10× one), which is why a big MoE can be worth trying at the edge of your tier.
- **Fitting beats fancy.** A 14B that fits will beat a 70B that spills, every day, on everything, because you'll actually use it.

Step 3 — the tier table

Speeds are **ballpark generation rates at Q4**, gathered from public benchmarks and rounded hard. Your runtime, quant, context length, and card generation all move them. Treat them as order-of-magnitude, then measure your own (Step 4).

your VRAM	sweet spot	example models to pull	ballpark speed
none (CPU only)	3–4B	`llama3.2:3b`, small qwen/nemotron nano builds	5–20 tok/s — slow but honest
6–8 GB	7–8B	`llama3.1:8b` (~4.9 GB), `mistral:7b`	30–70 tok/s
12 GB	12–14B	`qwen2.5:14b` (~9 GB)	25–50 tok/s
16 GB	~24B	`mistral-small:24b`, `devstral:24b` (~14 GB)	25–45 tok/s
24 GB	~32B, or fast mid-size MoE	`qwen3:32b` (~20 GB), `gpt-oss:20b` (MoE, quick)	30–45 tok/s dense; MoE much faster
32 GB	30–49B with headroom	32B-class + long context	45–70 tok/s
48 GB	70B	`llama3.3:70b` (~42 GB Q4)	15–30 tok/s
96 GB	100–120B-class MoE, or 70B at high precision	`gpt-oss:120b` (~65 GB), GLM-4.x-Air-class MoE, 70B Q8	~50 tok/s for the 120B MoE

Notes that save real money and time:

- **Tool calling matters later in this course** (Part 9). Check a model's page on ollama.com for tool support before making it your daily driver. `llama3.1:8b` supports it, which is why it's the course model.
- **Context eats VRAM too.** Leave headroom; a model that "just fits" with no room for context will spill the moment the conversation gets long.
- **Licenses ride along.** If a copy of your setup ever leaves your house — sold, gifted, deployed — the model's license applies to that copy. Apache-2.0 models (e.g. Mistral 7B) are the simple case; check each model's license page before shipping anything. For private use on your own box, pull whatever you like.

Step 4 — never trust a benchmark table, including this one

Ollama will tell you your real number:

```
ollama run llama3.1:8b --verbose
```

Ask it anything, then read the stats it prints at the end. Expected: several lines, the one that matters is `eval rate: XX tokens/s` — that's YOUR generation speed, on YOUR card, and it outranks every table on the internet. While it runs, `ollama ps` in another window shows whether the model is `100% GPU` or spilling (`xx%/yy% CPU/GPU`) — if it spills, step down a tier and feel the difference.

Step 5 — pull the course model

```
ollama pull llama3.1:8b
```

Expected: several progress bars (it downloads in layers), ~4.9 GB total, then "success". Small enough for almost any GPU, good general quality, supports tool calling. If your card is bigger, also pull something from your tier and use it as your daily model — but run the course exercises on `llama3.1:8b` so the listings match what you see.

First contact — no code yet:

```
ollama run llama3.1:8b
```

Expected: a `>>>` prompt. Type something; it answers, streaming word by word. Type `/bye` to leave. You now have a working local AI and have written zero code.

The moment that matters — prove the API exists. In PowerShell:

```
Invoke-RestMethod -Uri http://127.0.0.1:11434/api/chat -Method Post -Body '{"model":"llama3.1:8b
  ","messages":[{"role":"user","content":"say hi in five words"}],"stream":false}' -ContentType
  "application/json"
```

Expected: a JSON response whose `message` field contains the model's reply. Stare at this one. Your entire client is this call in a loop with better manners.

PART 4 — The flagships: what a big open model actually takes

The tier table stops at "what fits." This part walks two **flagship open-weight models** end to end — what they are, what hardware they really need, and every step from empty disk to reading your own speed number. Both are MoE (Part 1's vocabulary, worth restating): instead of one dense brain where every parameter works on every word, an MoE model is split into many "expert" blocks with a router that activates only a few per word. You still need memory for ALL the experts, but each word only pays for a few — big-model knowledge at mid-size-model speed. That trade is what makes flagships runnable at home at all.

Flagship 1 — gpt-oss:120b (OpenAI, Apache-2.0)

fact	number
Total / active parameters	~117B total, ~5B active per word (per the model card)
Download size	65 GB (ships natively quantized at ~4.25 bits, MXFP4)
Runs entirely in VRAM on	an 80 GB GPU (OpenAI's own line); 96 GB = comfortable with long context
Hybrid GPU+RAM	16–24 GB VRAM + 64 GB system RAM: works, expect a fraction of the speed
Ballpark speed	~50 tok/s all-in-VRAM on a 96 GB card; low single digits to teens when split
Little sibling	`gpt-oss:20b` — 14 GB, same family, fits a 16 GB card whole

Getting it running, start to finish:

1. **Check disk first.** You need 65+ GB free on the drive Ollama stores models on. `ollama list` shows what you already have; delete dead weight with `ollama rm NAME` — each removal prints "deleted" and frees the space immediately.
2. **Decide which sibling fits your card** with Part 3's fitting rule before downloading anything. 16 GB card -> the 20b. 80–96 GB card -> the 120b. In between -> the 20b whole beats the 120b split, and you can still try the 120b hybrid later.
3. **Pull it.** `ollama pull gpt-oss:120b` (or `:20b`). Expected: layered progress bars for a long while — this is a 65 GB download, plan around your internet, not the tool.
4. **First run.** `ollama run gpt-oss:120b --verbose` — expected: a pause of up to a few minutes while 65 GB loads into memory (first load is the slow one), then the `>>>` prompt. Ask it something real.
5. **Look at where it landed.** In a second window: `ollama ps` — expected: one row showing `100% GPU` (it fits) or a split like `40%/60% CPU/GPU` (it spilled). No guessing: this line is the truth about your hardware.
6. **Read your number.** The `--verbose` stats after each answer include `eval rate: XX tokens/s`. That is your real speed. Above ~20 you have a daily driver; below ~5 you have a curiosity.
7. **If it's a curiosity, reclaim the disk.** `ollama rm gpt-oss:120b` — expected: "deleted", 65 GB back. Keeping an unusable model "just in case" is how drives fill.

Flagship 2 — GLM-4.5-Air (Z.ai, MIT license)

fact	number
Total / active parameters	106B total, 12B active per word
Download size	~66 GB at Q4 (smaller, rougher quants exist down to ~30 GB)
Runs entirely in VRAM on	~67 GB needed at Q4 with modest context -> an 80–96 GB card
Hybrid GPU+RAM	24 GB VRAM + 64–96 GB system RAM: workable, MoE absorbs some of the pain
Character	more active parameters per word than gpt-oss — heavier per token, a different quality/speed trade

Getting it running:

1. **Same disk check** — ~66 GB free for the Q4.
2. **Check the Ollama library first** (ollama.com, search "glm"). If it's listed, the steps are identical to Flagship 1 with the listed name.
3. **If it's not listed, Ollama can run models straight off Hugging Face.** Find the model's GGUF page (search "GLM-4.5-Air GGUF"), pick the quantization your card affords by Part 3's fitting rule (Q4_K_M is the standard pick), and run `ollama run hf.co/UPLOADER/GLM-4.5-Air-GGUF:Q4_K_M` — the `hf.co/...` path IS the model name. Expected: same layered download, then the prompt. This one skill — running any GGUF by its Hugging Face path — unlocks every open model ever published, not just what a library curates.
4. **Verify the same way every time:** `ollama ps` for the GPU/CPU split, `--verbose` for your eval rate, `ollama rm` if the number says no.

The two, side by side

	gpt-oss:120b	GLM-4.5-Air
License	Apache-2.0	MIT
Download	65 GB	~66 GB (Q4)
All-in-VRAM needs	80 GB+	~67 GB+
Active params per word	~5B	12B
Feel	fast for its size, strong reasoning	heavier per word, strong all-rounder

The lesson both teach: **a flagship's requirement is not a GPU model name — it's "where does the whole file sit."** All in VRAM: flagship speed. Split with system RAM: an MoE stays usable where a dense model of the same size would crawl. On disk only: it doesn't run. Find your tier in Part 3's table, then spend your download budget on the biggest thing that sits where speed lives.

Numbers checked 2026-08-03 against ollama.com/library/gpt-oss and public GLM-4.5-Air spec pages; model sizes and library listings drift, so trust the model's own page over this table.

PART 5 — Your first client (~25 lines, stdlib only)

Make `C:\learn_ai\chat1.py`:

```
import json
import urllib.request

OLLAMA = "http://127.0.0.1:11434"   # loopback: never leaves this PC
MODEL = "llama3.1:8b"

def ask(prompt):
    payload = {
        "model": MODEL,
```

```
        "messages": [{"role": "user", "content": prompt}],
        "stream": False,
    }
    req = urllib.request.Request(
        OLLAMA + "/api/chat",
        data=json.dumps(payload).encode("utf-8"),
        headers={"Content-Type": "application/json"},
    )
    with urllib.request.urlopen(req, timeout=300) as resp:
        reply = json.loads(resp.read())
    return reply["message"]["content"]

print(ask("Explain what an API is in two sentences."))
```

Run it: `python chat1.py`. Expected: a two-sentence answer prints, then the program exits.

How to READ this listing (a skill this course teaches on purpose):

- Line 1–2: imports. Both from the standard library — this program has zero dependencies, which is why it can be copied to any machine as a bare file.
- `OLLAMA`/`MODEL` at the top: constants live at the top of a file so the knobs are visible without reading the machinery. When you open any new file, read the constants block first — it tells you what the file can be told to do.
- `payload`: a Python dict mirroring the JSON you sent by hand in Part 3. Same three fields: which brain, the conversation so far, stream or not.
- `urllib.request.Request(...)`: builds the HTTP POST. `.encode("utf-8")` because HTTP carries bytes, not strings.
- `json.loads(resp.read())`: the reverse — bytes back to a dict.
- `reply["message"]["content"]`: walking into the JSON to the one string you care about. When exploring a new API, `print(reply)` first and look at the whole shape — that's how you find these paths yourself.

PART 6 — Make it a conversation: the messages list

The model is **stateless**. It remembers nothing between calls. The "memory" of a chat is just you re-sending the whole conversation every single turn. Three roles exist: `system` (standing orders), `user` (the human), `assistant` (the model's own past replies).

`chat2.py`:

```
import json
import urllib.request

OLLAMA = "http://127.0.0.1:11434"
MODEL = "llama3.1:8b"

messages = [
    {"role": "system",
     "content": "You are a plain-spoken assistant. Short answers. No flattery."}
]

def call(messages):
    payload = {"model": MODEL, "messages": messages, "stream": False}
    req = urllib.request.Request(
        OLLAMA + "/api/chat",
        data=json.dumps(payload).encode("utf-8"),
        headers={"Content-Type": "application/json"},
    )
    with urllib.request.urlopen(req, timeout=300) as resp:
        return json.loads(resp.read())["message"]
```

```
while True:
    user = input("you > ").strip()
    if user in ("/exit", ""):
        break
    messages.append({"role": "user", "content": user})
    msg = call(messages)
    messages.append(msg)                # the model's reply joins the history
    print("ai  >", msg["content"])
```

Run it. Tell it your name, then ask what your name is. It knows — because the first exchange rode along in `messages` on the second call. Now `/exit`, rerun, ask again. It has no idea. **That's statelessness made visible.**

Cost of this design: the list grows every turn, and it must fit the context window. That's why real clients have a `/reset` (throw the list away, keep the system prompt) and why long sessions eventually need it.

PART 7 — Streaming (why it feels alive)

`"stream": true` makes Ollama send the reply as it's generated: one small JSON object per line, each carrying a few characters, until one says `"done": true`. Replace `call()` with:

```
def call_streaming(messages):
    payload = {"model": MODEL, "messages": messages, "stream": True}
    req = urllib.request.Request(
        OLLAMA + "/api/chat",
        data=json.dumps(payload).encode("utf-8"),
        headers={"Content-Type": "application/json"},
    )
    content = ""
    with urllib.request.urlopen(req, timeout=300) as resp:
        for raw_line in resp:                    # HTTP body, line by line
            chunk = json.loads(raw_line)
            piece = chunk.get("message", {}).get("content", "")
            print(piece, end="", flush=True)       # print as it arrives
            content += piece
            if chunk.get("done"):
                break
    print()
    return {"role": "assistant", "content": content}
```

Expected when you run it: the answer types itself out live instead of arriving in one block. `flush=True` matters — without it Python buffers the output and the "live" effect dies.

PART 8 — Persona

Make `persona.txt` next to your script, put standing orders in it (tone, rules, what it should refuse), and load it:

```
with open("persona.txt", "r", encoding="utf-8") as fh:
    messages = [{"role": "system", "content": fh.read()}]
```

Edit the file, restart, and the personality changes with zero code changes. One honest limit to know: **a system prompt is INSTRUCTIONS, not enforcement.** A model can be argued out of instructions. Real rules (what files can be touched, what runs) must live in code — which is the next part.

PART 9 — Hands (tool calling): the part that matters most

The concept in one paragraph. Along with your messages, you send a `tools` list — JSON descriptions of functions you're offering ("there is a function called list_folder; it takes a path"). If the model wants one, it doesn't answer in text: it replies with a `tool_calls` field naming the function and arguments. **The model has now done nothing.** It has asked. Your code decides whether to run anything, runs it if allowed, appends the result as a `{"role": "tool"}` message, and calls the model again — now it answers using what the tool returned. The model proposes; your dispatcher disposes. Every guard you will ever build lives in that dispatcher, which is why no amount of clever prompting can widen what the hands can do.

`hands1.py` — a chat client with ONE read-only hand and a jail:

```
import json
import os
import urllib.request

OLLAMA = "http://127.0.0.1:11434"
MODEL = "llama3.1:8b"
JAIL = os.path.realpath(r"C:\learn_ai\sandbox")   # the ONE folder hands may touch

TOOLS = [{
    "type": "function",
    "function": {
        "name": "list_folder",
        "description": "List the files in a folder inside the working area.",
        "parameters": {
            "type": "object",
            "properties": {
                "path": {"type": "string",
                         "description": "Folder path to list"}},
            "required": ["path"],
        },
    },
}]

def run_hand(name, args):
    """The dispatcher. THE security boundary. The model never gets past here."""
    if name != "list_folder":
        return "REFUSED: no such hand."
    target = os.path.realpath(str(args.get("path", "")))  # resolves .. and links
    if not target.startswith(JAIL):                       # the jail check
        return "REFUSED: that path is outside the working folder."
    try:
        return "\n".join(sorted(os.listdir(target))) or "(empty)"
    except OSError as e:
        return "ERROR: %s" % e

def call(messages):
    payload = {"model": MODEL, "messages": messages,
               "tools": TOOLS, "stream": False}
    req = urllib.request.Request(
        OLLAMA + "/api/chat",
        data=json.dumps(payload).encode("utf-8"),
        headers={"Content-Type": "application/json"},
    )
    with urllib.request.urlopen(req, timeout=300) as resp:
        return json.loads(resp.read())["message"]

os.makedirs(JAIL, exist_ok=True)
messages = [{"role": "system", "content":
             "You have a list_folder tool for the working folder. "
             "Use it rather than guessing what files exist."}]

while True:
    user = input("you > ").strip()
    if user in ("/exit", ""):
```

```
            break
        messages.append({"role": "user", "content": user})
        while True:                                  # loop: model may chain tools
            msg = call(messages)
            messages.append(msg)
            calls = msg.get("tool_calls")
            if not calls:
                print("ai  >", msg["content"])
                break
            for tc in calls:
                fn = tc["function"]
                print("[hand] %s(%s)" % (fn["name"], fn["arguments"]))
                result = run_hand(fn["name"], fn["arguments"])
                messages.append({"role": "tool", "content": result})
```

Try it: drop a few files in `C:\learn_ai\sandbox`, run, ask "what files are in the working folder?" Expected: a `[hand] list_folder(...)` line prints, then the model answers with the real names. Then ask it to list `C:\Windows`. Expected: the hand line prints, the dispatcher REFUSES, and the model relays the refusal. **You just watched a jail hold.**

Read the two safety lines like a security engineer:

- `os.path.realpath()` BEFORE the check — resolves `..`, junctions, and symlinks first, so `C:\learn_ai\sandbox\..\..\Windows` can't sneak past a naive string check. Resolve first, THEN compare. Order is everything.
- The check refuses and returns a STRING — it never raises to the model, never half-runs. Fail closed, explain plainly.

Where's the write side? Deliberately not here. Hands that CHANGE files deserve more than a paragraph — preview-first rehearsal, a backup of anything they touch, and a consent gate that stops and waits for a human before a single byte moves. That discipline is taught live and hands-on at VetTech Homefront's free community workshops — dates on the site. Bring a laptop with this part's build on it and you'll leave with write hands you gated yourself. Everything read-only in this course is complete as printed; nothing above was held back.

PART 10 — The GUI: putting a window on it

Tkinter's one law: the `mainloop()` thread owns the window. Block that thread (say, waiting 30 seconds for a model), and the window freezes gray. So every GUI chat client has the same skeleton:

1. A **worker thread** talks to the model.
2. It pushes text into a **queue** (thread-safe mailbox).
3. The GUI thread **polls** the queue every 50 ms with `after()` and drains it into the window.

`gui1.py` — minimal windowed chat:

```
import json
import queue
import threading
import tkinter as tk
import urllib.request
from tkinter import scrolledtext

OLLAMA = "http://127.0.0.1:11434"
MODEL = "llama3.1:8b"
Q = queue.Queue()
messages = [{"role": "system", "content": "Short, plain answers."}]

def worker(user_text):
    messages.append({"role": "user", "content": user_text})
    payload = {"model": MODEL, "messages": messages, "stream": True}
```

```
        req = urllib.request.Request(
            OLLAMA + "/api/chat",
            data=json.dumps(payload).encode("utf-8"),
            headers={"Content-Type": "application/json"})
        content = ""
        try:
            with urllib.request.urlopen(req, timeout=300) as resp:
                for line in resp:
                    chunk = json.loads(line)
                    piece = chunk.get("message", {}).get("content", "")
                    content += piece
                    Q.put(piece)                    # -> the GUI thread
                    if chunk.get("done"):
                        break
            messages.append({"role": "assistant", "content": content})
        except OSError as e:
            Q.put("\n[error talking to Ollama: %s]" % e)
        Q.put("\n\n")

    win = tk.Tk()
    win.title("my first ai window")
    txt = scrolledtext.ScrolledText(win, wrap="word", state="disabled")
    txt.pack(fill="both", expand=True)
    entry = tk.Entry(win)
    entry.pack(fill="x")

    def append(s):
        txt.configure(state="normal")
        txt.insert("end", s)
        txt.see("end")
        txt.configure(state="disabled")

    def on_send(_event=None):
        user = entry.get().strip()
        if not user:
            return
        entry.delete(0, "end")
        append("you > %s\nai  > " % user)
        threading.Thread(target=worker, args=(user,), daemon=True).start()

    def poll():
        try:
            while True:
                append(Q.get_nowait())
        except queue.Empty:
            pass
        win.after(50, poll)                         # re-arm the poll

    entry.bind("<Return>", on_send)
    win.after(50, poll)
    entry.focus_set()
    win.mainloop()
```

Run `python gui1.py`. Expected: a window; type, press Enter, and the reply streams into the pane while the window stays responsive — because the model call is on the worker thread and only the queue crosses over.

PART 11 — How to read a codebase (the method, in order)

1. **Read the header docstring.** Good files declare their shape and their rules up front.
2. **Find the entry point.** Search `if __name__ ==` — that block calls `main()`. Read `main()` top to bottom once, shallow: it's the table of contents (parse args, build state, start loop).

3. **Follow ONE thing end to end.** For a chat client: one user message, from `input()` to the printed reply. Ignore every branch you don't hit. First-pass understanding is a single lit path, not the whole map.
4. **Grep before you scroll.** Looking for a consent gate? Search `YES`. Model switching? Search the variable name. Read the DEFINITION first, then its callers.
5. **Constants and env reads are the knobs.** A line like `MODEL = os.environ.get("MY_MODEL", ...)` tells you there's a lever, its name, and its default — without reading any logic.
6. **Comments earn trust only about WHY.** When a comment says what the code does and disagrees with the code, the code is the truth. When it says why ("resolve BEFORE compare, because links"), that's the load-bearing part.
7. **Break something on purpose** (on a COPY). Change the jail check to `if False`, watch the refusal vanish, revert. Nothing teaches what a line does like removing it.

PART 12 — The safety patterns, and what each one stops

Every one of these lives in this course at toy scale — in print above, or in the workshop half (Part 9's note). In a client you use for real work, they are not optional decorations — each exists because of a specific failure it prevents.

pattern	you built it in	what it stops
Loopback-only, checked in code	Parts 5–10 (`OLLAMA` constant)	your prompts ever leaving the box; a poisoned env var redirecting the client
Jail: realpath + prefix, refuse-first	Part 9 `run_hand`	any hand touching anything outside the ONE folder, including via `..`/links
Typed YES, exact, default-refuse	taught live at the workshops	reflex-clicking a permission; consent without reading
Preview free, apply gated	taught live at the workshops	irreversible surprise; you always see the exact change first
Read-only mode = write tools ABSENT	taught live at the workshops	the model even being OFFERED a write until a human flips the switch
Backup before write, diff from bytes	taught live at the workshops	trusting the model's account of what it changed; unrecoverable edits
Fail closed, explain plainly	Part 9 dispatcher	half-run operations; errors the human never sees

The deeper rule behind all of them: **a capability that is off is not "forbidden" — it is absent.** The model cannot be talked into using a tool your code never offered. Subtraction beats supervision.

PART 13 — Exercises (do them on a COPY, in order)

1. **Change the model without code.** Make your script read `os.environ.get("MY_MODEL", "llama3.1:8b")`, set the env var, confirm it switched. (Proves: Part 11 point 5.)
2. **Add `/time`.** In chat2.py, make `/time` print the local time without calling the model. (Proves: commands are just an `if` before the model call.)
3. **Add a read-only hand** `word_count(path)` to hands1.py: jail check, read, return the count. Ask the model to use it. (Proves: registry + dispatcher.)
4. **Attack your own jail.** Ask the model to list `..\..` and `C:\Windows`. Watch every one refuse. (Proves: the guard — and that "the model tried" is not an event. Your dispatcher was the only thing that ever mattered.)

The write-side exercises — a gated write hand, preview/apply, backup-before-write, and the typed-YES consent discipline — are the workshop half of this course (see Part 9's note). The room is where you build those with someone checking your gates.

Built and published by VetTech Homefront, a veteran-owned local-first AI business in Ohio. Companion repo: a finished, free, chat-only client built on exactly these patterns.